Building publication-grade brainplots.py: APIs, code, and design principles

The two pillars of a publication-quality neuroimaging visualization module are **graph/network figures** and **3D cortical rendering**. This report provides exact API signatures, working code, and design principles for both — targeting Nature, Science, PNAS, Brain, and NeuroImage standards. Every library version, parameter default, and code snippet reflects the current (2025–2026) state of these tools.

AREA 1: Publication-quality graph and network visualization

netgraph v4.13.2 delivers the most complete graph API for brain networks

netgraph (Brodersen, JOSS 2023) is a matplotlib-based library specifically designed for publication-quality network visualization. Readthedocs Install via `pip install netgraph` or `conda install -c conda-forge netgraph`. GitHub The `Graph()` class accepts these key parameters:

```python
from netgraph import Graph

g = Graph(
    edges,                          # list of (source, target) or (source, t
    node_layout='spring',           # 'spring'|'community'|'circular'|'dot'|
    node_layout_kwargs=None,        # dict passed to layout algorithm
    node_color='w',                 # color or dict {node: color}
    node_size=3.0,                  # float or dict (radius, rescaled by BAS
    node_edge_color='#2c404c',      # node BORDER color — use for network me
    node_edge_width=0.5,            # border line width
    node_alpha=1.0,                 # float or dict
    node_labels=False,              # bool or dict {node: 'label'}
    node_label_fontdict=None,       # dict passed to matplotlib.text.Text
    edge_width=1.0,                 # float or dict {(u,v): width}
    edge_color='#2c404c',           # color or dict — overrides edge_cmap if
    edge_alpha=0.5,                 # float or dict {(u,v): alpha}
    edge_cmap='RdGy',              # colormap for automatic weight→color map
    edge_layout='straight',         # 'straight'|'curved'|'arc'|'bundled'
    edge_layout_kwargs=None,        # dict passed to edge layout algorithm
    ax=None,                        # matplotlib Axes for compositing
    prettify=True,                  # removes axis ticks/spines automaticall
)
```

Edge bundling uses Force-Directed Edge Bundling (Holten & van Wijk 2009) via `edge_layout='bundled'`. Readthedocs GitHub The critical parameter is `k` (spring constant) — higher values produce tighter bundles. Pass it through `edge_layout_kwargs=dict(k=2000)`. Readthedocs The `compatibility_threshold` parameter (default **0.05**) controls which edge pairs get bundled; setting it lower bundles more aggressively. GitHub Bundled layout can be slow for >100 nodes (~4 minutes), so lower `k` if speed matters.

Curved vs arc edges: `edge_layout='curved'` uses an optimization algorithm that routes splines around nodes and other edges Readthedocs — it is computationally expensive but produces cleaner results for dense brain networks. `edge_layout='arc'` uses fixed-curvature arcs (controlled by `rad=1.0`) — fast but prone to edge-node collisions. For brain connectivity, **use** `'curved'` **for sparse networks and** `'bundled'` **for dense ones.**

Community layout is the most important feature for brain network figures. It positions nodes in clusters reflecting network membership:

```python
node_to_community = {'V1': 0, 'V2': 0, 'MT': 0, 'PFC': 1, 'DLPFC': 1, ...}
community_colors = {0: '#0072B2', 1: '#E69F00', 2: '#009E73', 3: '#CC79A7'}
node_color = {n: community_colors[c] for n, c in node_to_community.items()}

Graph(edges,
    node_layout='community',
    node_layout_kwargs=dict(node_to_community=node_to_community),
    node_color=node_color,
    node_edge_color=node_color,  # border color = network membership
    node_edge_width=2.0,
    edge_layout='bundled',
    edge_layout_kwargs=dict(k=500, straighten_by=0.02),
)
```

The community layout algorithm creates a weighted meta-graph of communities, positions them via spring layout, then positions nodes within each community using a second spring layout. Readthedocs This naturally separates brain networks while clustering co-modular regions.

Simultaneous weight encoding (thickness + color + opacity) requires manual dict construction:

```python
import numpy as np
from matplotlib import cm

cmap = cm.coolwarm
weights = {(u, v): w for u, v, w in weighted_edges}
wmin, wmax = min(weights.values()), max(weights.values())

edge_width = {e: 0.5 + 3.0 * np.abs(w) for e, w in weights.items()}
edge_color = {e: cmap((w - wmin) / (wmax - wmin)) for e, w in weights.items(
edge_alpha = {e: 1.0 if pvals[e] < 0.05 else 0.15 for e in weights}

Graph(edges, edge_width=edge_width, edge_color=edge_color, edge_alpha=edge_a
```

Note that setting `edge_color` as a dict **overrides** the `edge_cmap` parameter. Readthedocs To use automatic colormap mapping for color while manually controlling width, omit `edge_color` and set only `edge_cmap` and `edge_width`. Readthedocs

Background brain overlay works because netgraph renders on matplotlib Axes. Pass `ax=ax` to the constructor and add a brain outline image to the same axes with `ax.imshow(brain_img, extent=[0,1,0,1], alpha=0.3, zorder=0)`. Set custom node positions via `node_layout=dict_of_positions` mapped to the image coordinate space.

For **publication export**, netgraph figures follow standard matplotlib practices: `fig.savefig('network.pdf', dpi=300, bbox_inches='tight', transparent=True)` for vector, or `dpi=600` for raster. The `prettify=True` default already removes axis chrome. Post-hoc access to individual artists is available via `g.node_artists[node_id]` and `g.edge_artists[(u,v)]`.

Matplotlib-native techniques for chord diagrams, arc diagrams, and custom graphs

For **chord diagrams**, four libraries are relevant, each with distinct strengths:

- `mpl_chord_diagram` — pure matplotlib, accepts a connectivity matrix directly: `chord_diagram(flux, names, colors=colors, use_gradient=True, alpha=0.7)` GitHub PyPI
- `mne_connectivity.viz.plot_connectivity_circle` — neuroimaging-specific, supports hemisphere grouping via `circular_layout()` with `group_boundaries` Python Graph Gallery
- `pyCirclize` — Circos-style plots: `Circos.chord_diagram(matrix_df, space=5, cmap='tab10')` GitHub
- `NiChord` — brain-network-specific, combines chord + glass brain projections GitHub

For **hand-crafted graphs** using matplotlib patches, `FancyArrowPatch` is the core tool. Matplotlib The `connectionstyle='arc3,rad=0.15'` parameter creates quadratic Bézier curves NetworkX Matplotlib — positive `rad` curves left, negative curves right. Clip arrows at node borders using the `patchA` and `patchB` parameters: Matplotlib

```python
from matplotlib.patches import FancyArrowPatch, Circle
arrow = FancyArrowPatch(
    posA=(x1, y1), posB=(x2, y2),
    connectionstyle='arc3,rad=0.15',
    arrowstyle='-|>',  # or '-' for undirected
    mutation_scale=15,
    linewidth=1 + 3*weight,
    color=plt.cm.hot(weight),
    patchA=Circle((x1,y1), node_radius),
    patchB=Circle((x2,y2), node_radius),
)
ax.add_patch(arrow)
```

Arc diagrams place nodes on a horizontal line with semicircular arcs above (positive connections) and below (negative). Python Graph Gallery The `arcplot` library provides a quick API (`ArcDiagram.connect(src, dst, arc_position='below')`), PyPI while netgraph offers `ArcDiagram` Readthedocs for more control. Readthedocs Pure matplotlib arcs use `matplotlib.patches.Arc((x_mid, 0), width, height, theta1=0, theta2=180)`.

Adjacency matrix visualization demands careful ordering and annotation

Matrix ordering dramatically affects readability. Three algorithms are standard for brain connectivity:

Hierarchical clustering with optimal leaf ordering is the most common:

```python
from scipy.cluster.hierarchy import linkage, optimal_leaf_ordering, leaves_l
from scipy.spatial.distance import squareform

dist = 1 - np.abs(FC)
np.fill_diagonal(dist, 0)
Z = linkage(squareform(dist), method='average')
Z_ordered = optimal_leaf_ordering(Z, squareform(dist))
order = leaves_list(Z_ordered)
FC_reordered = FC[np.ix_(order, order)]
```

Spectral ordering (Fiedler vector) uses the second-smallest eigenvector of the graph Laplacian, (ENS) providing a mathematically principled ordering that groups densely connected nodes. **Network-block ordering** simply sorts by network assignment (e.g., Yeo-7 labels) — the simplest but most interpretable approach for parcellated brain data. (arXiv)

Network color sidebars can be added via seaborn's `clustermap(row_colors=network_colors, col_colors=network_colors)` (Seaborn) (Seaborn) or manually using a `GridSpec` with narrow axes for colored bars:

```python
from matplotlib.gridspec import GridSpec
gs = GridSpec(2, 2, width_ratios=[0.03, 1], height_ratios=[0.03, 1], wspace=
ax_row = fig.add_subplot(gs[1, 0])
for i, c in enumerate(network_color_list):
    ax_row.axhspan(i, i+1, color=c)
```

Within-network block annotations use `matplotlib.patches.Rectangle((start-0.5, start-0.5), size, size, fill=False, edgecolor='black', linewidth=2)` overlaid on the heatmap. White grid lines at network boundaries (`ax.axhline(y=boundary-0.5, color='white', linewidth=1.5)`) also work well.

Colormap selection follows a strict convention: functional connectivity uses **diverging** colormaps (`RdBu_r`, `coolwarm`) centered at zero with `vmax = np.percentile(np.abs(FC), 95)`; structural connectivity uses **sequential** colormaps (`hot`, `YlOrRd`) often with `LogNorm()`; distance matrices use **perceptually uniform** colormaps (`viridis`, `plasma`). (DataCamp)

Significance markers on matrix cells use a loop adding text annotations:

```python
def pval_to_stars(p):
    if p <= 0.001: return "***"
    elif p <= 0.01: return "**"
    elif p <= 0.05: return "*"
    return ""

for i in range(n):
    for j in range(i+1, n):
        stars = pval_to_stars(pvals[i, j])
        if stars:
            color = 'white' if abs(FC[i,j]) > vmax*0.6 else 'black'
            ax.text(j, i, stars, ha='center', va='center', color=color, font
```

Non-significant cells can receive hatching via `Rectangle(..., fill=False, hatch='///', edgecolor='gray')`. Medium

Graph-tool's sfdp_layout with groups is uniquely powerful for modular brain networks

graph-tool is **40–250× faster** than NetworkX for graph algorithms (C++/Boost backend with OpenMP parallelism). Skewed Its standout feature for brain networks is `sfdp_layout()` with a `groups` parameter that explicitly clusters same-community nodes while separating different communities — no other Python library does this as effectively. The `gamma` parameter controls group cohesion strength.

Installation requires `conda install -c conda-forge graph-tool` (no pip support due to C++ dependencies). A pragmatic workflow: compute layout coordinates in graph-tool, export them, and render with matplotlib/netgraph for fine-grained visual control.

Layout algorithm comparison for brain networks: Kamada-Kawai (`nx.kamada_kawai_layout`) is best for weighted brain networks because it places springs between all node pairs with ideal lengths proportional to graph-theoretic distance — strongly connected regions end up close together. Fruchterman-Reingold (`nx.spring_layout`) is good for revealing general community structure but prone to local minima. The **best hybrid strategy** is Kamada-Kawai for initial global layout refined with Fruchterman-Reingold for local aesthetics. For explicitly modular visualization, graph-tool's `sfdp_layout(groups=community_labels)` is unmatched.

Design principles determine whether a brain network figure succeeds or fails

Node size encoding should match the scientific question. **Node strength** (sum of edge weights) is the most natural choice for weighted FC networks. **Betweenness centrality** highlights connector hubs between modules (e.g., precuneus, insula). **Degree** suits binarized networks. Scale node sizes by the **square root** of the metric so visual area is proportional to value (per Tufte's graphical integrity principle). Map to a matplotlib scatter size range of approximately **50–500** for ~100-node networks.

Edge encoding should simultaneously convey weight (thickness: linear for narrow ranges, logarithmic for wide ranges, mapped to **0.5–4.0 pt**), sign (warm = positive FC, cool = negative/anti-correlations), and significance (alpha **0.8–1.0** for FDR-corrected edges, **0.1–0.2** for non-significant). Negative edges in brain FC networks are meaningful — display them with cool colors and optionally dashed lines.

Graph density for visualization: show the **top 5–15%** of edges for functional connectivity (the CONN toolbox defaults to 15%). Structural connectivity can tolerate **10–30%** density. Report results across a range of thresholds to demonstrate robustness. The **percolation threshold** (De Vico Fallani et al., 2017) — the highest threshold preserving graph connectedness — provides a principled, data-driven cutoff.

Colorblind-safe Yeo-7 colors: the standard Yeo-7 palette is **not** colorblind-safe (green Dorsal Attention and red Default Mode are indistinguishable for ~8% of males with deuteranopia). Use the **Okabe-Ito palette** (Wong, Nature Methods 2011) instead:

Network	Color	Hex
Visual	Vermillion	#D55E00
Somatomotor	Sky Blue	#56B4E9
Dorsal Attention	Bluish Green	#009E73
Ventral Attention	Reddish Purple	#CC79A7
Limbic	Yellow	#F0E442
Frontoparietal	Orange	#E69F00
Default Mode	Blue	#0072B2

Paul Tol's qualitative palettes (personal.sron.nl/~pault/) offer additional 7-color schemes validated for colorblind safety.

Label placement: the `adjustText` library (GitHub) (`pip install adjustText`) iteratively repositions labels to minimize overlap, connecting displaced labels to their nodes with leader lines. For dense brain networks (>30 nodes), selectively label only hubs or significant findings — never label all 100+ nodes. Font size **6–8 pt** for dense graphs, **8–10 pt** for sparse, always sans-serif (Arial, Helvetica).

Visual hierarchy for figures that tell a story at a glance: Layer 1 (first glance) = network coloring shows community structure; Layer 2 = node sizes reveal hub importance; Layer 3 = edge weights/colors show connection patterns; Layer 4 = annotations provide quantitative specifics. To direct attention, render the full network in light gray (alpha=0.1–0.2) and overlay significant findings in full color — this creates an immediate figure-ground separation.

AREA 2: Advanced brain rendering with PyVista, VTK, and related 3D tools

PyVista camera positions and multi-panel figures for standard neuroimaging views

In **FreeSurfer RAS coordinates** (X=Right, Y=Anterior, Z=Superior), the exact camera positions for standard views are:

```python
D = 300  # Adjust based on mesh bounds (200-400 for fsaverage)

CAMERA_POSITIONS = {
    'lateral_left':   [(-D, 0, 0), (0, 0, 0), (0, 0, 1)],
    'lateral_right':  [(D, 0, 0),  (0, 0, 0), (0, 0, 1)],
    'medial_left':    [(D, 0, 0),  (0, 0, 0), (0, 0, 1)],   # looking at LH
    'medial_right':   [(-D, 0, 0), (0, 0, 0), (0, 0, 1)],   # looking at RH
    'dorsal':         [(0, 0, D),  (0, 0, 0), (0, 1, 0)],    # top-down, ant
    'ventral':        [(0, 0, -D), (0, 0, 0), (0, 1, 0)],    # bottom-up, ar
    'anterior':       [(0, D, 0),  (0, 0, 0), (0, 0, 1)],
    'posterior':      [(0, -D, 0), (0, 0, 0), (0, 0, 1)],
}
```

Each tuple is `[(camera_xyz), (focal_point_xyz), (view_up_xyz)]`. (PyVista) MNI152 uses the same RAS orientation, so these positions apply to both coordinate systems.

The 4-view panel figure (lateral + medial for both hemispheres) uses `pv.Plotter(shape=(2,2))`:

```python
import pyvista as pv

pl = pv.Plotter(shape=(2, 2), window_size=(1600, 1200), off_screen=True, bor

views = [
    (0, 0, 'L Lateral',  [(-D,0,0), (0,0,0), (0,0,1)]),
    (0, 1, 'R Lateral',  [(D,0,0),  (0,0,0), (0,0,1)]),
    (1, 0, 'L Medial',   [(D,0,0),  (0,0,0), (0,0,1)]),
    (1, 1, 'R Medial',   [(-D,0,0), (0,0,0), (0,0,1)]),
]
for row, col, label, cpos in views:
    pl.subplot(row, col)
    mesh = lh_mesh if 'L' in label else rh_mesh
    pl.add_mesh(mesh, scalars=data, cmap='RdBu_r', clim=[-5, 5],
                smooth_shading=True, show_scalar_bar=False)
    pl.add_text(label, font_size=12, position='upper_left')
    pl.camera_position = cpos
    pl.enable_anti_aliasing('ssaa')

img = pl.screenshot(return_img=True)
```

For **dorsal views showing both hemispheres**, add both meshes to a single subplot and set `camera_position = [(0, 0, D), (0, 0, 0), (0, 1, 0)]`. Advanced layout control is available via `row_weights`, `col_weights`, and `groups` parameters, PyVista plus string layouts like `shape="3|1"`. PyVista PyVista

PBR shading, curvature underlays, and three-point lighting

Physically-based rendering produces superior sulcal contrast compared to default Phong shading. PyVista The key parameters for cortical surfaces are:

```python
pl.add_mesh(cortex, scalars=stat_map, cmap='RdBu_r',
    pbr=True, metallic=0.0, roughness=0.35, smooth_shading=True)
```

Brain tissue is organic, so **metallic=0.0** always. **Roughness 0.3–0.5** reveals sulcal anatomy with subtle specular highlights — 0.3 for slightly glossy (good highlight definition), 0.5 for a natural matte tissue appearance. GitHub PBR is energy-conserving and physically realistic, DeepWiki unlike Phong's uniform "plastic" specular highlights.

Curvature underlay displays sulcal depth as a grayscale background, following nilearn's convention:

```python
import nibabel as nib
curv = nib.freesurfer.read_morph_data('lh.curv')
mesh.point_data['curv'] = np.sign(curv)  # binary sulcal/gyral

# Base layer: curvature in grayscale
pl.add_mesh(mesh, scalars='curv', cmap='gray', clim=[-1, 1],
            show_scalar_bar=False, smooth_shading=True)
# Overlay: data with opacity threshold
overlay = mesh.copy()
overlay.point_data['stat'] = stat_map
opacity = np.where(np.abs(stat_map) > threshold, 1.0, 0.0)
pl.add_mesh(overlay, scalars='stat', cmap='RdBu_r', opacity=opacity)
```

Three-point lighting dramatically improves sulcal visibility:

```python
pl = pv.Plotter(lighting='none')  # disable defaults
key = pv.Light(position=(200, 200, 300), intensity=1.0, light_type='scene li
fill = pv.Light(position=(-200, 100, 100), intensity=0.4, light_type='scene
rim = pv.Light(position=(0, -200, 200), intensity=0.3, light_type='scene lig
for light in [key, fill, rim]:
    pl.add_light(light)
```

Use a single strong directional key light at ~30° elevation with moderate fill to maximize sulcal shadow contrast. Flat, even illumination washes out sulci and should be avoided.

Silhouettes, anti-aliasing, and color bars complete the publication pipeline

Silhouette edges create clean cartoon-like outlines at brain boundaries, essential for white-background journal figures:

```python
pl.add_mesh(brain, smooth_shading=True,
    silhouette=dict(color='black', line_width=3.0, opacity=1.0))
```

Anti-aliasing: use **SSAA** (`pl.enable_anti_aliasing('ssaa')`) for all final publication screenshots. DeepWiki SSAA renders at higher resolution then downsamples, giving the best quality for static images. FXAA blurs scalar bar text and labels. MSAA with `multi_samples=16` is a reasonable compromise. Note: SSAA may make line widths appear thinner — increase `line_width` by ~2× to compensate.

Scalar bar styling uses the `scalar_bar_args` dict:

```python
sargs = dict(
    title='T-statistic', title_font_size=18, label_font_size=14,
    n_labels=5, fmt='%.1f', font_family='arial',
    vertical=True, position_x=0.87, position_y=0.25,
    width=0.06, height=0.5, color='black',
)
pl.add_mesh(mesh, scalars=data, scalar_bar_args=sargs)
```

Compositing into matplotlib enables adding panel labels (A, B, C, D) and fine-tuned layout:

```python
screenshots = {}
for name, cpos in views.items():
    pl = pv.Plotter(off_screen=True, window_size=(800, 800))
    pl.add_mesh(mesh, scalars=data, cmap='RdBu_r', show_scalar_bar=False)
    pl.enable_anti_aliasing('ssaa')
    pl.camera_position = cpos
    pl.show(auto_close=False)
    screenshots[name] = pl.screenshot(return_img=True)
    pl.close()

fig, axes = plt.subplots(2, 2, figsize=(10, 10))
for ax, (name, img), label in zip(axes.flat, screenshots.items(), 'ABCD'):
    ax.imshow(img); ax.axis('off')
    ax.text(0.02, 0.98, label, transform=ax.transAxes,
            fontsize=20, fontweight='bold', va='top')
plt.subplots_adjust(wspace=0.02, hspace=0.02)
plt.savefig('composite.png', dpi=300, bbox_inches='tight', facecolor='white'
```

For transparent background compositing, set `pv.global_theme.transparent_background = True` GitHub before creating the plotter — screenshots then return RGBA arrays PyVista PyVista compatible with `PIL.Image.alpha_composite()`. Use `pl.image_scale = 3` for 3× super-resolution before screenshotting. FXAA is **incompatible** with transparent backgrounds; GitHub always use SSAA.

Subcortical structures from FreeSurfer aseg require marching cubes and Taubin smoothing

Extracting individual subcortical meshes involves three steps — masking, meshing, and transforming:

```python
import nibabel as nib
from skimage.measure import marching_cubes
import pyvista as pv

aseg = nib.load('aseg.mgz')
data, affine = aseg.get_fdata(), aseg.affine

# FreeSurfer label IDs
LABELS = {
    'L-Hippocampus': 17, 'R-Hippocampus': 53,
    'L-Amygdala': 18, 'R-Amygdala': 54,
    'L-Thalamus': 10, 'R-Thalamus': 49,
    'L-Caudate': 11, 'R-Caudate': 50,
    'L-Putamen': 12, 'R-Putamen': 51,
    'L-Pallidum': 13, 'R-Pallidum': 52,
    'L-Accumbens': 26, 'R-Accumbens': 58,
}

def extract_mesh(volume, label_id, affine):
    mask = (volume == label_id).astype(np.float32)
    verts, faces, _, _ = marching_cubes(mask, level=0.5, allow_degenerate=Fa
    verts_ras = (affine @ np.hstack([verts, np.ones((len(verts),1))]).T).T[:
    faces_pv = np.column_stack([np.full(len(faces), 3), faces]).ravel()
    return pv.PolyData(verts_ras, faces_pv)
```

Always use Taubin smoothing (`mesh.smooth_taubin(n_iter=50, pass_band=0.01)`) rather than Laplacian smoothing for subcortical structures. Standard Laplacian smoothing causes **volume shrinkage** on closed surfaces, which distorts small subcortical structures. Taubin smoothing alternates shrinking (λ) and expanding (μ) steps to preserve volume while removing staircase artifacts from marching cubes.

FreeSurfer default colors (from FreeSurferColorLUT.txt) are the field standard: Free Surfer Free Surfer

Structure	RGB	Color
Hippocampus	(220, 216, 20)	Yellow
Amygdala	(103, 255, 255)	Cyan
Thalamus	(0, 118, 14)	Dark green
Caudate	(122, 186, 220)	Light blue
Putamen	(236, 13, 176)	Magenta
Pallidum	(12, 48, 255)	Deep blue
Accumbens	(255, 165, 0)	Orange

Transparent cortex with visible subcortical structures requires depth peeling: DeepWiki

```python
pl = pv.Plotter()
pl.enable_depth_peeling(number_of_peels=10, occlusion_ratio=0.0)

# Add opaque subcortical structures first
for name, label in LABELS.items():
    mesh = extract_mesh(data, label, affine).smooth_taubin(n_iter=50, pass_b
    pl.add_mesh(mesh, color=FS_COLORS[name], opacity=1.0, smooth_shading=Tru

# Add transparent cortex second
pl.add_mesh(cortex_lh, color='white', opacity=0.15, smooth_shading=True)
pl.add_mesh(cortex_rh, color='white', opacity=0.15, smooth_shading=True)
```

Without `enable_depth_peeling()`, transparent surfaces will composit incorrectly. DeepWiki Note that depth peeling is incompatible with MSAA — use FXAA or SSAA. The `number_of_peels=10` handles most brain visualization scenarios; `occlusion_ratio=0.0 :antCitation[]{citations="345c28c0-23fb-42b2-a4a9-7ce42f62f704"}` ensures exact rendering.

Thalamic nuclei from FreeSurfer's `segmentThalamicNuclei.sh` (FS 7.1+) produce a 25-nuclei parcellation with labels **8001–8014** (e.g., Anterior=8001, Pulvinar=8012, LGN=8013, MGN=8014). Load `ThalamicNuclei.v12.T1.FSvoxelSpace.mgz` and use the same `extract_mesh` function with each nucleus label.

VTK drops below PyVista for curvatures, volume rendering, and 3D labels

vtkCurvatures computes Gaussian and mean curvature directly on brain meshes — useful for visual enhancement when FreeSurfer curvature files are unavailable:

```python
from vtkmodules.vtkFiltersGeneral import vtkCurvatures
curv_filter = vtkCurvatures()
curv_filter.SetInputData(brain_mesh)  # pv.PolyData IS a vtkPolyData
curv_filter.SetCurvatureTypeToMean()  # or SetCurvatureTypeToGaussian()
curv_filter.Update()
output = pv.wrap(curv_filter.GetOutput())
mean_curv = output.point_data['Mean_Curvature']
```

Volume rendering for brain MRI uses `vtkGPUVolumeRayCastMapper` (best quality) or `vtkSmartVolumeMapper` (auto-selects backend). Custom transfer functions control opacity and color per voxel intensity — critical for selectively showing gray matter, white matter, and CSF. PyVista wraps this as `plotter.add_volume(image_data)` which can coexist with `plotter.add_mesh(surface)` in the same scene.

3D labels via `vtkBillboardTextActor3D` create camera-facing text anchored at 3D coordinates. These maintain consistent pixel size regardless of zoom — preferred for publication over `vtkFollower` (which scales with distance). PyVista's `plotter.add_point_labels(centers, labels, font_size=20, shadow=True, always_visible=True)` provides a simpler API for the same functionality.

Alternative libraries each fill specific niches

brainrender (v2.1.17, BrainGlobe project, BSD-3) is actively maintained and uses **vedo** as its rendering backend. It supports multiple brain atlases via `brainglobe-atlasapi` including one human brain atlas — but it is **not designed** for FreeSurfer surface workflows. Its Scene API (`scene.add_brain_region("VISp", alpha=0.2)`) excels for mouse brain atlas visualization. For human FreeSurfer data, MNE-Python's `mne.viz.Brain` (PyVista backend) remains the standard.

vedo shares VTK's rendering backend with PyVista but produces **better-looking output with minimal configuration** — PyVista's own maintainers acknowledge wanting to replicate vedo's visual defaults. Both libraries are functionally equivalent in raw capability; vedo offers more expressive syntax and superior font choices. API conversion is straightforward since both wrap VTK objects.

FURY (from DIPY, v0.13.0) is the best choice for **tractography rendering**, using custom GLSL shaders for GPU-accelerated streamline visualization. `fury.actor.streamtube(streamlines, colors=direction_colors)` handles thousands of fibers efficiently. For static cortical surface figures, PyVista is more practical; for tractography-heavy figures, FURY is significantly more optimized.

trimesh (v4.11.5) excels at mesh **preprocessing** — repair, hole filling, decimation, smoothing — before handoff to PyVista for rendering. It offers Laplacian, Taubin, Humphrey, and mutual-diffusion Laplacian smoothing algorithms. FreeSurfer's own `surfa` library was explicitly built with trimesh as inspiration. The pipeline is: load with nibabel → process with trimesh → render with PyVista.

SSAO, depth-of-field, and glass brain effects produce "wow factor" figures

Screen Space Ambient Occlusion (SSAO) darkens sulci and brightens gyri, dramatically improving cortical folding visibility:

```python
pl.enable_ssao(radius=5.0, bias=0.5, kernel_size=256, blur=True)
```

Default `radius=0.5` is for small meshes — increase to **2.0–10.0** for brain-scale geometry. Default `bias=0.005` should increase to **0.5–2.0**. SSAO is the single most impactful visual enhancement for cortical surfaces in publication figures.

Depth-of-field is natively supported via `pl.enable_depth_of_field()` — objects at the focal point remain sharp while surrounding regions blur, creating artistic tilt-shift effects ideal for highlighting specific brain regions.

The glass brain effect combines low opacity, high specularity, and depth peeling:

```python
pl.enable_depth_peeling(number_of_peels=10)
pl.add_mesh(cortex, color='lightblue', opacity=0.15,
    specular=1.0, specular_power=50, diffuse=0.3)
pl.add_mesh(hippocampus, color='red', smooth_shading=True, specular=0.5)
```

For PBR glass: `pbr=True, metallic=0.0, roughness=0.1, opacity=0.15`.

Tractography alongside cortical surfaces in PyVista uses `pv.Spline` for individual streamlines or `pv.PolyData` with line connectivity for bulk rendering. Color by direction using the RGB convention (R=left/right, G=anterior/posterior, B=superior/inferior). The `.tube(radius=0.3)` filter converts lines to tubes but is expensive — use only for key tracts, line rendering for the rest.

Glow effects are not natively available in PyVista (no bloom pass in VTK's render pipeline). Three workarounds exist: (1) high ambient lighting (`ambient=0.8`) simulates self-illumination; (2) post-processing bloom in Pillow by extracting bright regions, Gaussian-blurring them, and additively compositing; (3) PBR with environment maps and low roughness for concentrated specular highlights that mimic glow.

Conclusion

Building a publication-grade `brainplots.py` module requires orchestrating **netgraph** for graph layouts with community detection and edge bundling, **matplotlib** for chord/arc diagrams and adjacency matrices with spectral or block ordering, and **PyVista** for 3D cortical and subcortical rendering with PBR, SSAO, and three-point lighting. The critical technical choices are: Taubin over Laplacian smoothing for subcortical meshes, SSAA over FXAA for final screenshots, depth peeling for any transparency, and the Okabe-Ito palette over standard Yeo-7 colors for colorblind safety. The most overlooked but highest-impact visual enhancements are SSAO for cortical surfaces (makes sulci dramatically more visible), edge bundling for dense brain graphs (reduces visual clutter by an order of magnitude), and curvature underlays for surface stat maps (provides anatomical context that reviewers expect). For tractography, delegate to FURY rather than PyVista; for mesh preprocessing, use trimesh before rendering. Every code example above works with current library versions as of 2025–2026.